

Security and Hardening Guide

Applies to SUSE Linux Enterprise Server 15 SP4

23 Masquerading and firewalls <#>

Whenever Linux is used in a network environment, you can use the kernel functions that allow the manipulation of network packets to maintain a separation between internal and external network areas. The Linux `netfilter` framework provides the means to establish an effective firewall that keeps different networks apart. Using `iptables`—a generic table structure for the definition of rule sets—precisely controls the packets allowed to pass a network interface. Such a packet filter can be set up using `firewalld` and its graphical interface `firewall-config`.

SUSE Linux Enterprise Server 15 GA introduces `firewalld` as the new default software firewall, replacing `SuSEfirewall2`. This chapter provides guidance for configuring `firewalld`, and migrating from `SuSEfirewall2` for users who have upgraded from older SUSE Linux Enterprise Server releases.

23.1 Packet filtering with `iptables` <#>

This section discusses the low-level details of packet filtering. The components `netfilter` and `iptables` are responsible for the filtering and manipulation of network packets and for network address translation (NAT). The filtering criteria and any actions associated with them are stored in chains, which must be matched one after another by individual network packets as they arrive. The chains to match are stored in tables. The `iptables` command allows you to alter these tables and rule sets.

The Linux kernel maintains three tables, each for a particular category of functions of the packet filter:

`filter`

This table holds the bulk of the filter rules, because it implements the *packet filtering* mechanism in the stricter sense, which determines whether packets are let through (ACCEPT) or discarded (DROP), for example.

`nat`

This table defines any changes to the source and target addresses of packets. Using these functions also allows you to implement *masquerading*, which is a special case of NAT used to link a private network with the Internet.

mangle

The rules held in this table make it possible to manipulate values stored in IP headers (such as the type of service).

These tables contain several predefined chains to match packets:

PREROUTING

This chain is applied to all incoming packets.

INPUT

This chain is applied to packets destined for the system's internal processes.

FORWARD

This chain is applied to packets that are only routed through the system.

OUTPUT

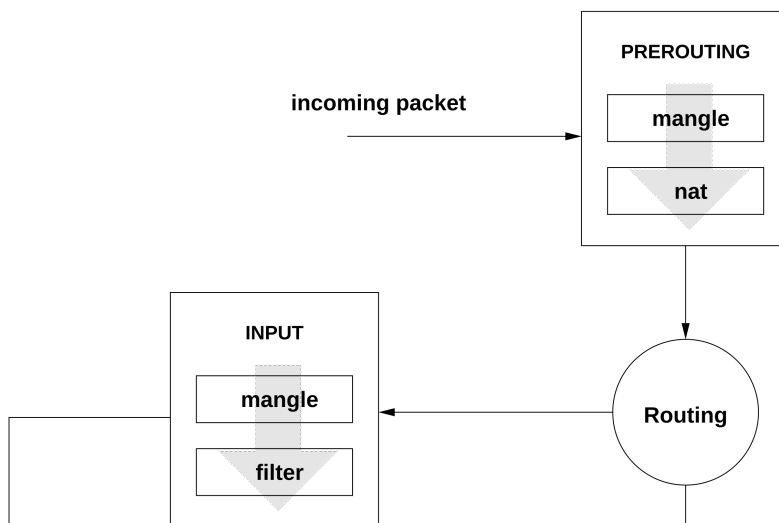
This chain is applied to packets originating from the system itself.

POSTROUTING

This chain is applied to all outgoing packets.

[Figure 23.1, “iptables: a packet's possible paths”](#) illustrates the paths along which a network packet may travel on a given system. For the sake of simplicity, the figure lists tables as parts of chains, but in reality these chains are held within the tables themselves.

In the simplest case, an incoming packet destined for the system itself arrives at the `eth0` interface. The packet is first referred to the PREROUTING chain of the `mangle` table then to the PREROUTING chain of the `nat` table. The following step, concerning the routing of the packet, determines that the actual target of the packet is a process of the system itself. After passing the INPUT chains of the `mangle` and the `filter` table, the packet finally reaches its target, provided that the rules of the `filter` table allow this.



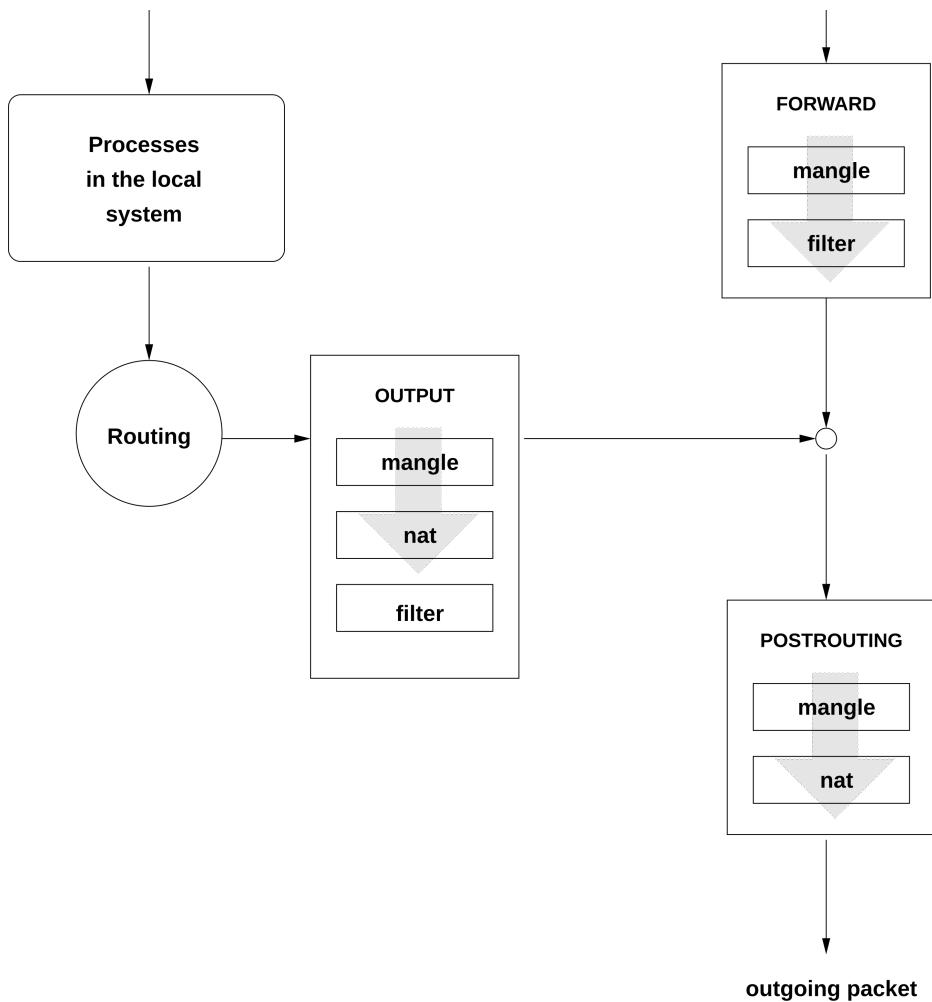


Figure 23.1: iptables: a packet's possible paths <#>

23.2 Masquerading basics <#>

Masquerading is the Linux-specific form of NAT (network address translation) and can be used to connect a small LAN with the Internet. LAN hosts use IP addresses from the private range (see [Section 23.1.2, “Netmasks and routing”](#)) and on the Internet official IP addresses are used. To be able to connect to the Internet, a LAN host's private address is translated to an official one. This is done on the router, which acts as the gateway between the LAN and the Internet. The underlying principle is a simple one: The router has more than one network interface, typically a network card and a separate interface connecting with the Internet. While the latter links the router with the outside world, one or several others link it with the LAN hosts. With these hosts in the local network connected to the network card (such as `eth0`) of the router, they can send any packets not destined for the local network to their default gateway or router.



Important: Using the correct network mask

When configuring your network, make sure both the broadcast address and the netmask are the same for all local hosts. Failing to do so prevents packets from being routed properly.

As mentioned, whenever one of the LAN hosts sends a packet destined for an Internet address, it goes to the default router. However, the router must be configured before it can forward such packets. For security reasons, this is not enabled in a default installation. To enable it, add the line `net.ipv4.ip_forward = 1` in the file `/etc/sysctl.conf`. Alternatively do this via YaST, for example by calling `yast routing ip-forwarding on`.

The target host of the connection can see your router, but knows nothing about the host in your internal network where the packets originated. This is why the technique is called masquerading. Because of the address translation, the router is the first destination of any reply packets. The router must identify these incoming packets and translate their target addresses, so packets can be forwarded to the correct host in the local network.

With the routing of inbound traffic depending on the masquerading table, there is no way to open a connection to an internal host from the outside. For such a connection, there would be no entry in the table. In addition, any connection already established has a status entry assigned to it in the table, so the entry cannot be used by another connection.

As a consequence of all this, you might experience some problems with several application protocols, such as ICQ, cueme, IRC (DCC, CTCP), and FTP (in PORT mode). Web browsers, the standard FTP program, and many other programs use the PASV mode. This passive mode is much less problematic as far as packet filtering and masquerading are concerned.

23.3 Firewalling basics <#>

Firewall is probably the term most widely used to describe a mechanism that controls the data flow between networks. Strictly speaking, the mechanism described in this section is called a *packet filter*. A packet filter regulates the data flow according to certain criteria, such as protocols, ports, and IP addresses. This allows you to block packets that, according to their addresses, are not supposed to reach your network. To allow public access to your Web server, for example, explicitly open the corresponding port. However, a packet filter does not scan the contents of packets with legitimate addresses, such as those directed to your Web server. For example, if incoming packets were intended to compromise a CGI program on your Web server, the packet filter would still let them through.

A more effective but more complex mechanism is the combination of several types of systems, such as a packet filter interacting with an application gateway or proxy. In this case, the packet filter rejects any packets destined for disabled ports. Only packets directed to the application gateway are accepted. This gateway or proxy pretends to be the actual client of the server. In a sense, such a proxy could be considered a masquerading host on the protocol level used by the application. One example

for such a proxy is Squid, an HTTP and FTP proxy server. To use Squid, the browser must be configured to communicate via the proxy. Any HTTP pages or FTP files requested are served from the proxy cache and objects not found in the cache are fetched from the Internet by the proxy.

The following section focuses on the packet filter that comes with SUSE Linux Enterprise Server. For further information about packet filtering and firewalling, read the [Firewall HOWTO](#).

23.4 firewalld



Note: firewalld replaces SuSEfirewall2

SUSE Linux Enterprise Server 15 GA introduces firewalld as the new default software firewall, replacing SuSEfirewall2. If you are upgrading from a release older than SUSE Linux Enterprise Server 15 GA, SuSEfirewall2 will be unchanged and you must manually upgrade to firewalld (see [Section 23.5, “Migrating from SuSEfirewall2”](#)).

firewalld is a daemon that maintains the system's iptables rules and offers a D-Bus interface for operating on them. It comes with a command line utility `firewall-cmd` and a graphical user interface `firewall-config` for interacting with it. Since firewalld is running in the background and provides a well defined interface it allows other applications to request changes to the iptables rules, for example to set up virtual machine networking.

firewalld implements different security zones. Several predefined zones like `internal` and `public` exist. The administrator can define additional custom zones if desired. Each zone contains its own set of iptables rules. Each network interface is a member of exactly one zone. Individual connections can also be assigned to a zone based on the source addresses.

Each zone represents a certain level of trust. For example the `public` zone is not trusted, because other computers in this network are not under your control (suitable for Internet or wireless hotspot connections). On the other hand the `internal` zone is used for networks that *are* under your control, like a home or company network. By utilizing zones this way, a host can offer different kinds of services to trusted networks and untrusted networks in a defined way.

For more information about the predefined zones and their meaning in firewalld, refer to its manual at <http://www.firewalld.org/documentation/zone/predefined-zones.html>.



Note: No zone assigned behavior

The initial state for network interfaces is to be assigned to no zone at all. In this case the network

interface will be implicitly handled in the default zone, which can be determined by calling `firewall-cmd --get-default-zone`. If not configured otherwise, the default zone is the public zone.

The `firewalld` packet filtering model allows any outgoing connections to pass. Outgoing connections are connections that are actively established by the local host. Incoming connections that are established by remote hosts are blocked if the respective service is not allowed in the zone in question. Therefore, each of the interfaces with incoming traffic must be placed in a suitable zone to allow for the desired services to be accessible. For each of the zones, define the services or protocols you need.

An important concept of `firewalld` is the distinction between two separate configurations: the *runtime* and the *permanent* configuration. The runtime configuration represents the currently active rules, while the permanent configuration represents the saved rules that will be applied when restarting `firewalld`. This allows to add temporary rules that will be discarded after restarting `firewalld`, or to experiment with new rules while being able to revert back to the original state. When you are changing the configuration, you need to be aware of which configuration you are editing. How this is done is discussed in [Section 23.4.3.2, “Runtime versus permanent configuration”](#).

To perform the `firewalld` configuration using the graphical user interface `firewall-config` refer to its [documentation](#). In the following section we will be looking at how to perform typical `firewalld` configuration tasks using `firewall-cmd` on the command line.

23.4.1 Configuring the firewall with NetworkManager <#>

The NetworkManager supports a basic configuration of `firewalld` by selecting zones.

When editing a wired or wireless connection, go to the *Identity* tab in the configuration window and use the Firewall Zone drop-down box.

23.4.2 Configuring the Firewall with YaST <#>

The `yast firewall` module supports a basic configuration of `firewalld`. It provides a zone selector, services selector, and ports selector. It does not support creating custom iptables rules, and limits zone creation and customization to selecting services and ports.



Note: Changing settings in running mode

YaST respects the settings in `/etc/firewalld/firewalld.conf`, where the default value for `FlushAllOnReload` is set to `no`. Therefore, YaST does not change settings in running mode. For example, if you have assigned an interface to a different zone with YaST, restart the `firewalld` daemon for the change to take effect.

23.4.3 Configuring the firewall on the command line <#>

23.4.3.1 Firewall start-up <#>

firewalld will be installed and enabled by default. It is a regular systemd service that can be configured via systemctl or the YaST Services Manager.



Important: Automatic firewall configuration

After the installation, YaST automatically starts firewalld and leaves all interfaces in the default public zone. If a server application is configured and activated on the system, YaST can adjust the firewall rules via the options or in the server configuration modules. Some server module dialogs include a button for activating additional services and ports.

23.4.3.2 Runtime versus permanent configuration <#>

By default all firewall-cmd commands operate on the runtime configuration. You can apply most operations to the permanent configuration *only* by adding the --permanent parameter. When doing so the change will only affect the permanent configuration and will not be effective immediately in the runtime configuration. There is currently no way to add a rule to both runtime and permanent configurations in a single invocation. To achieve this you can apply all necessary changes to the runtime configuration and when all is working as expected issue the following command:

```
#
```

```
firewall-cmd --runtime-to-permanent
```

This will write all current runtime rules into the permanent configuration. Any temporary modifications you or other programs may have made to the firewall in other contexts are made permanent this way. If you are unsure about this, you can also take the opposite approach to be on the safe side: Add new rules to the permanent configuration and reload firewalld to make them active.



Note

Some configuration items, like the default zone, are shared by both the runtime and permanent configurations. Changing them will reflect in both configurations at once.

To revert the runtime configuration to the permanent configuration and thereby discard any temporary changes, two possibilities exist, either via the firewalld command line interface or via

systemd:

```
#
```

```
firewall-cmd --reload
```

```
#
```

```
systemctl reload firewalld
```

For brevity the examples in the following sections will always operate on the runtime configuration, if applicable. Adjust them accordingly to make them permanent.

23.4.3.3 Assignment of interfaces to zones <#>

You can list all network interfaces currently assigned to a zone like this:

```
#
```

```
firewall-cmd --zone=public --list-interfaces
```

eth0

Similarly you can query which zone a specific interface is assigned to:

```
#
```

```
firewall-cmd --get-zone-of-interface=eth0
```

public

The following command lines assign an interface to a zone. The variant using `--add-interface` will only work if `eth0` is not already assigned to another zone. The variant using `--change-interface` will always work, removing `eth0` from its current zone if necessary:

```
#
```

```
firewall-cmd --zone=internal --add-interface=eth0
```

```
#
```

```
firewall-cmd --zone=internal --change-interface=eth0
```

Any operations without an explicit `--zone` argument will implicitly operate on the default zone. This pair of commands can be used for getting and setting the default zone assignment:

```
#
```

```
firewall-cmd --get-default-zone
```

```
dmz
```

```
#
```

```
firewall-cmd --set-default-zone=public
```



Important

Any network interfaces not explicitly assigned to a zone will be automatically part of the default zone. Changing the default zone will reassign all those network interfaces immediately for the permanent and runtime configurations. You should never use a trusted zone like `internal` as the default zone, to avoid unexpected exposure to threats. For example hotplugged network interfaces like USB Ethernet interfaces would automatically become part of the trusted zone in such cases.

Also note that interfaces that are not explicitly part of any zone will not appear in the zone interface list. There is currently no command to list unassigned interfaces. Due to this it is best to avoid unassigned network interfaces during regular operation.

23.4.3.4 Making network services accessible <#>

`firewalld` has a concept of *services*. A service consists of definitions of ports and protocols. These definitions logically belong together in the context of a given network service like a Web or mail server protocol. The following commands can be used to get information about predefined services and their details:

```
#
```

```
firewall-cmd --get-services
```

```
[...] dhcp dhcpv6 dhcpv6-client dns docker-registry [...]
```

```
#
```

```
firewall-cmd --info-service dhcp
```

```
dhcp
```

```
ports: 67/udp
```

protocols:
source-ports:
modules:
destination:

These service definitions can be used for easily making the associated network functionality accessible in a zone. This command line will open the HTTP Web server port in the internal zone, for example:

```
#  
firewall-cmd --add-service=http --zone=internal
```

The removal of a service from a zone is performed using the counterpart command `--remove-service`. You can also define custom services using the `--new-service` subcommand. Refer to <http://www.firewalld.org/documentation/howto/add-a-service.html> for more details on how to do this.

If you just want to open a single port by number, you can use the following approach. This will open TCP port 8000 in the internal zone:

```
#  
firewall-cmd --add-port=8000/tcp --zone=internal
```

For removal use the counterpart command `--remove-port`.



Tip: Temporarily opening a service or port

`firewalld` supports a `--timeout` parameter that allows to open a service or port for a limited time duration. This can be helpful for quick testing and makes sure that closing the service or port will not be forgotten. To allow the `imap` service in the internal zone for 5 minutes, you would call

```
#  
firewall-cmd --add-service=imap --zone=internal --timeout=5m
```

23.4.3.5 Lockdown mode <#>

`firewalld` offers a *lockdown mode* that prevents changes to the firewall rules while it is active. Since applications can automatically change the firewall rules via the D-Bus interface, and depending on the PolicyKit rules regular users may be able to do the same, it can be helpful to prevent changes in some situations. You can find more information about this at <https://fedoraproject.org/wiki/Features/FirewalldLockdown>.

It is important to understand that the lockdown mode feature provides no real security, but merely protection against accidental or benign attempts to change the firewall. The way the lockdown mode is currently implemented in `firewalld` provides no security against malicious intent, as is pointed out at <http://seclists.org/oss-sec/2017/q3/139>.

23.4.3.6 Adding custom iptables rules <#>

`firewalld` claims exclusive control over the host's netfilter rules. You should never modify firewall rules using other tools like `iptables`. Doing so could confuse `firewalld` and break security or functionality.

If you need to add custom firewall rules that aren't covered by `firewalld` features then there are two ways to do so. To directly pass raw `iptables` syntax you can use the `--direct` option. It expects the table, chain, and priority as initial arguments and the rest of the command line is passed as is to `iptables`. The following example adds a connection tracking rule for the forwarding filter table:

```
#  
firewall-cmd --direct --add-rule ipv4 filter FORWARD 0 -i eth0 -o eth1 \  
-p tcp --dport 80 -m state --state NEW,RELATED,ESTABLISHED -j ACCEPT
```

Additionally, `firewalld` implements so called *rich rules*, an extended syntax for specifying `iptables` rules in an easier way. You can find the syntax specification at <http://www.firewalld.org/documentation/man-pages/firewalld.richlanguage.html>. The following example drops all IPv4 packets originating from a certain source address:

```
#  
firewall-cmd --zone=public --add-rich-rule='rule family="ipv4" \  
source address="192.168.2.4" drop'
```

23.4.3.7 Routing, forwarding, and masquerading <#>

`firewalld` is not designed to run as a fully fledged router. The basic functionality for typical home router setups is available. For a corporate production router you should not use `firewalld`, however, but use dedicated router and firewall devices instead. The following provides just a few pointers on what to look for to utilize routing in `firewalld`:

First of all IP forwarding needs to be enabled as outlined in [Section 23.2, “Masquerading basics”](#).

To enable IPv4 masquerading, for example in the external zone, issue the following command.

```
#
```

```
firewall-cmd --zone=external --add-masquerade
```

firewalld can also enable port forwarding. The following command will forward local TCP connections on port 80 to another host:

```
#  
firewall-cmd --zone=public \  
--add-forward-port=port=80:proto=tcp:toport=80:toaddr=192.168.1.10
```

23.4.4 Accessing services listening on dynamic ports <#>

Some network services do not listen on predefined port numbers. Instead they operate based on the portmapper or rpcbind protocol. We will use the term rpcbind from here on. When one of these services starts, it chooses a random local port and talks to rpcbind to make the port number known. rpcbind itself is listening on a well known port. Remote systems can then query rpcbind about the network services it knows about and on which ports they are listening. Not many programs use this approach anymore today. Popular examples are Network Information Services (NIS; ypserv and ypbind) and the Network File System (NFS) version 3.

The dynamic nature of the rpcbind protocol makes it difficult to make the affected services behind the firewall accessible. firewalld does not support these services by itself. For manual configuration, see [Section 23.4.4.1, “Configuring static ports”](#). Alternatively, SUSE Linux Enterprise Server provides a helper script. For details, see [Section 23.4.4.2, “Using firewall-rpcbind-helper for configuring static ports”](#).

23.4.4.1 Configuring static ports <#>

One possibility is to configure all involved network services to use fixed port numbers. Once this is done, the fixed ports can be opened in firewalld and everything should work. The actual port numbers used are at your discretion but should not clash with any well known port numbers assigned to other services. See [Table 23.1, “Important sysconfig variables for static port configuration”](#) for a list of the available configuration items for NIS and NFSv3 services. Note that depending on your actual NIS or NFS configuration, not all of these ports may be required for your setup.

Table 23.1: Important sysconfig variables for static port configuration <#>

File Path	Variable Name	Example Value
/etc/sysconfig/nfs	MOUNTD_PORT	21001
	STATD_PORT	21002

File Path	Variable Name	Example Value
	LOCKD_TCPPORT	21003
	LOCKD_UDPSPORT	21003
	RQUOTAD_PORT	21004
/etc/sysconfig/ypbind	YPBIND_OPTIONS	-p 24500
/etc/sysconfig/ypserv	YPXFRD_ARGS	-p 24501
	YPSERV_ARGS	-p 24502
	YPPASSWDD_ARGS	--port 24503

You will need to restart any related services that are affected by these static port configurations for the changes to take effect. You can see the currently assigned rpcbind ports by using the command `rpcinfo -p`. On success only the statically configured ports should show up there.

Apart from the port configuration for network services running in userspace there are also ports that are used by the Linux kernel directly when it comes to NFS. One of these ports is `nfs_callback_tcpport`. It is only required for NFS protocol versions older than 4.1. There is a sysctl named `fs.nfs.nfs_callback_tcpport` to configure this port. This sysctl node only appears dynamically when NFS mounts are active. Therefore it is best to configure the port via kernel module parameters. This can be achieved by creating a file as shown in [Example 23.1, “Callback port configuration for the nfs kernel module in /etc/modprobe.d/60-nfs.conf”](#).

Example 23.1: Callback port configuration for the nfs kernel module in `/etc/modprobe.d/60-nfs.conf` <#>

```
options nfs callback_tcpport=21005
```

To make this change effective it is easiest to reboot the machine. Otherwise all NFS services need to be stopped and the nfs kernel module needs to be reloaded. To verify the active NFS callback port, check the output of `cat /sys/module/nfs/parameters/callback_tcpport`.

For easy handling of the now statically configured RPC ports, it is useful to create a new `firewalld` service definition. This service definition will group all related ports and, for example, makes it easy to make them accessible in a specific zone. In [Example 23.2, “Commands to define a new firewalld RPC service for NFS”](#) this is done for the NFS ports as they have been configured in the accompanying examples.

Example 23.2: Commands to define a new `firewalld` RPC service for NFS <#>

```
#
```

```
firewall-cmd --permanent --new-service=nfs-rpc
```

```
#
```

```
firewall-cmd --permanent --service=nfs-rpc --set-description="NFS related,  
statically configured RPC ports"
```

```
# add UDP and TCP ports for the given sequence
```

```
#
```

```
for port in 21001 21002 21003 21004; do  
    firewall-cmd --permanent --service=nfs-rpc --add-port ${port}/udp --add-port  
${port}/tcp  
done
```

```
# the callback port is TCP only
```

```
#
```

```
firewall-cmd --permanent --service=nfs-rpc --add-port 21005/tcp
```

```
# show the complete definition of the new custom service
```

```
#
```

```
firewall-cmd --info-service=nfs-rpc --permanent -v
```

nfs-rpc

summary:

description: NFS and related, statically configured RPC ports

ports: 4711/tcp 21001/udp 21001/tcp 21002/udp 21002/tcp 21003/udp 21003/tcp 21004/
udp 21004/tcp

protocols:

source-ports:

modules:

destination:

```
# reload firewalld to make the new service definition available
```

```
#
```

```
firewall-cmd --reload
```

the new service definition can now be used to open the ports for example in the internal zone

```
#
```

```
firewall-cmd --add-service=nfs-rpc --zone=internal
```

23.4.4.2 Using firewall-rpcbind-helper for configuring static ports <#>

The steps to configure static ports as shown in the previous section can be simplified by using the SUSE helper tool `firewall-rpc-helper.py`. Install it with `zypper` in `firewalld-rpcbind-helper`.

The tool allows interactive configuration of the service patterns discussed in the previous section. It can also display current port assignments and can be used for scripting. For details, see `firewall-rpc-helper.py --help`.

23.4.4.3 nftables as the default back-end <#>

The default back-end for `firewalld` is `nftables`. `nftables` is a framework by the Linux `netfilter` project that provides packet filtering, network address translation (NAT) and other similar functionalities. `nftables` reuses the existing `Netfilter` subsystems such as the connection tracking system, user space queueing, logging and the hook infrastructure. `nftables` is a replacement for `iptables`, `ip6tables`, `arptables`, `ebtables`, and `ipset`.

The advantages of using `nftables` include:

One framework for both the IPv4 and IPv6 protocols.

Rules are applied atomically instead of fetching, updating and storing a complete rule set.

Monitor trace events in the `nft` tool, debug and trace the ruleset via `nfttrace`.

The `nft` command line tool compiles into VM bytecode in `netlink` format. During the rule set retrieval, the VM bytecode in `netlink` format is translated back into the original ruleset representation.

A basic nftables configuration file:

```
>
```

```
cat /etc/nftables.conf
```

```
#!/usr/sbin/nft -f
```

```
flush ruleset
```

```
# This matches IPv4 and IPv6
```

```
table inet filter {  
    # chain names are up to you.  
    # what part of the traffic they cover,  
    # depends on the type table.  
    chain input {  
        type filter hook input priority 0; policy accept;  
    }  
    chain forward {  
        type filter hook forward priority 0; policy accept;  
    }  
    chain output {  
        type filter hook output priority 0; policy accept;  
    }  
}
```

In the above example, all traffic is allowed because all chains have the policy *accept*. Note that, in reality, this is a bad practice, as firewalls should deny traffic by default.

A basic nftables configuration file with masquerade:

```
>
```

```
cat /etc/nftables.conf
```

```
#!/sbin/nft -f
```

```
flush ruleset
```

```
table ip nat {  
    chain prerouting {  
        type nat hook prerouting priority 0; policy accept;  
    }  
}
```

```
    # for all packets to WAN, after routing, replace source address with primary  
    IP of WAN interface
```

```

chain postrouting {
    type nat hook postrouting priority 100; policy accept;
    oifname "wan0" masquerade
}
}

```

If you have a static IP, it would be slightly faster to use source nat (SNAT) instead of masquerade. The router replaces the source with a predefined IP instead of looking up the outgoing IP for every packet.

An nftables configuration file with some rules applied:

```
>
```

```

cat /etc/nftables.conf
#!/usr/sbin/nft -f

```

```
flush ruleset
```

```

table inet filter {
    chain base_checks {
        ## another set, this time for connection tracking states.
        # allow established/related connections
        ct state {established, related} accept;

        # early drop of invalid connections
        ct state invalid drop;
    }
}

```

```

chain input {
    type filter hook input priority 0; policy drop;

    # allow from loopback
    iif "lo" accept;

    jump base_checks;

    # allow icmp and igmp
    ip6 nexthdr icmpv6 icmpv6 type { echo-request, echo-reply, packet-too-big,
time-exceeded, parameter-problem, destination-unreachable, packet-too-big, mld-

```

```

listener-query, mld-listener-report, mld-listener-reduction, nd-router-solicit, nd-
router-advert, nd-neighbor-solicit, nd-neighbor-advert, ind-neighbor-solicit, ind-
neighbor-advert, mld2-listener-report } accept;
    ip protocol icmp icmp type { echo-request, echo-reply, destination-
unreachable, router-solicitation, router-advertisement, time-exceeded, parameter-
problem } accept;
    ip protocol igmp accept;

    # for testing reject with logging
    counter log prefix "[nftables] input reject " reject;
}
chain forward {
    type filter hook forward priority 0; policy accept;
}
chain output {
    type filter hook output priority 0; policy accept;
}
}

```

In the above example:

The chain policy is to drop and add a reject rule at the end, which means all incoming traffic is blocked.

All traffic on the localhost interface is allowed.

The `base_checks` chain handles all packets that are related to established connections. This ensures that incoming packets for outgoing connections are not blocked.

ICMP and IGMP packets are allowed by utilizing a set and specific type names.

A counter keeps a count of both the total number of packets and bytes it has seen since it was last reset. With nftables, you must specify a counter for each rule you want to count.

23.4.4.3.1 More information #

For more information about nftables, see:

[Upstream documentation](#)

[nftables man page](#)

23.5 Migrating from SuSEfirewall2 <#>



Note: Creating a firewalld configuration for AutoYaST

See the *Firewall Configuration* section of the AutoYaST Guide to learn how to create a firewalld configuration for AutoYaST.

When upgrading from any service pack of SUSE Linux Enterprise Server 12 to SUSE Linux Enterprise Server 15 SP4, SuSEfirewall2 is not changed and remains active. There is no automatic migration, so you must migrate to firewalld manually. firewalld includes a helper migration script, `susefirewall2-to-firewalld`. Depending on the complexity of your SuSEfirewall2 configuration, the script may perform a perfect migration, or it may fail. Most likely it will partially succeed and you will have to review your new firewalld configuration and make adjustments.

The resulting configuration will make firewalld behave somewhat like SuSEfirewall2. To take full advantage of firewalld's features you may elect to create a new configuration, rather than trying to migrate your old configuration. It is safe to run the `susefirewall2-to-firewalld` script with no options, as it makes no permanent changes to your system. However, if you are administering the system remotely you could get locked out.

Install and run `susefirewall2-to-firewalld`:

```
#
```

```
zypper in susefirewall2-to-firewalld
```

```
#
```

```
susefirewall2-to-firewalld
```

```
INFO: Reading the /etc/sysconfig/SuSEfirewall2 file
```

```
INFO: Ensuring all firewall services are in a well-known state.
```

```
INFO: This will start/stop/restart firewall services and it's likely
```

```
INFO: to cause network disruption.
```

```
INFO: If you do not wish for this to happen, please stop the script now!
```

```
5...4...3...2...1...Lets do it!
```

```
INFO: Stopping firewalld
```

```
INFO: Restarting SuSEfirewall2_init
```

```
INFO: Restarting SuSEfirewall2
```

```
INFO: DIRECT: Adding direct rule="ipv4 -t filter -A INPUT -p udp -m udp --dport 5353  
-m pkttype
```

```
--pkt-type multicast -j ACCEPT"
[...]
INFO: Enabling direct rule=ipv6 -t filter -A INPUT -p udp -m udp --dport 546 -j
ACCEPT
INFO: Enabling direct rule=ipv6 -t filter -A INPUT -p udp -m udp --dport 5353 -m
pkttype
--pkt-type multicast -j ACCEPT
INFO: Enable logging for denied packets
INFO: #####
INFO:
INFO: The dry-run has been completed. Please check the above output to ensure
INFO: that everything looks good.
INFO:
INFO: #####
INFO: Stopping firewalld
INFO: Restarting SuSEfirewall2_init
INFO: Restarting SuSEfirewall2
```

This results in a lot of output, which you may wish to direct to a file for easier review:

```
#
```

```
susefirewall2-to-firewalld | tee newfirewallrules.txt
```

The script supports these options:

- c
Commit changes. The script will make changes to the system, so make sure you only use this option if you are really happy with the proposed changes. This *will* reset your current firewalld configuration, so make sure you make backups!
- d
Super noisy. Use it to file bug reports but be careful to mask sensitive information.
- h
This message.
- q
No output. Errors will not be printed either!
- v
Verbose mode. It will print warnings and other informative messages.

23.6 More information <#>

The most up-to-date information and other documentation about the `firewalld` package is found in `/usr/share/doc/packages/firewalld`. The home page of the netfilter and iptables project, <http://www.netfilter.org>, provides a large collection of documents about iptables in general in many languages.